

Workforce Identity / Employment Architecture

Reference Catalogue and Package Guide

This PDF explains the purpose, contents, and correct usage of the packaged Markdown files that define the Workforce identity, employee-record, linkage, RBAC, lifecycle, API, service, and rollout architecture.

Primary use cases:

- update the master plan
- hand work to Copilot or another coding agent
- align backend schema and services
- align frontend access-context bootstrap
- preserve field ownership and tenant safety during implementation

Core architecture rule: User accounts are independent platform identities; employee profiles are employer-controlled workforce records; active linkage between them is what activates scoped workforce permissions for the authenticated user.

Package generated for repository planning and implementation handoff.

How to Use This Package

Use the Markdown files as the implementation source material and the PDF as the human-readable navigation guide. The files are organized so that you can move from conceptual architecture to schema, then to services, endpoints, tests, and rollout sequencing without losing the separation between user identity and employer-controlled employee data.

Recommended reading order:

Order	File	Why read it
1	01_ARCHITECTURAL_RULE.md	Fast alignment on the non-negotiable model rule.
2	02_IDENTITY_EMPLOYEE_ACCESS_MODEL.md	Conceptual identity / employee / link separation.
3	03_CONCRETE_DATA_MODEL_AND_BACKEND_SCHEMA.md	Entity schema direction and permission model.
4	09_RBAC_INTEGRATION_AND_EFFECTIVE_PERMISSIONS.md	Permissions and roles overview.
5	10_EMPLOYEE_LIFECYCLE_TRANSITIONS_AND_ACCESS_EFFECTS.md	How state changes affect permissions and history.
6	11_SELF_SERVICE_VS_EMPLOYER_MANAGED_SELF_SERVICE.md	Ownership domains.
7	14_API_CONTRACT_AND_ENDPOINT_INVENTORY.md	API endpoint surface for frontend/backend work.
8	16_SERVICE_LAYER_ARCHITECTURE_AND_AUTHORIZATION_HEADERS.md	Service layer architecture.
9	17_CONCRETE_IMPLEMENTATION_ROADMAP.md	Safe build order.

Correct usage pattern:

- 1. Use the architectural rule and consolidated spec to align decisions.
- 2. Use the schema, API, RBAC, lifecycle, and service files as implementation references.
- 3. Use the Copilot prompt and execution sequence for small patch-based repo work.
- 4. Use the roadmap and master-plan sections to populate or refine your phased build list.

Package Inventory

README.md

Top-level package overview and folder guide.

markdown/00_PACKAGE_INDEX.md

Quick navigation map and recommended read order.

markdown/01_ARCHITECTURAL_RULE.md

One-page rule that defines the identity/employment separation.

markdown/02_IDENTITY_EMPLOYEE_ACCESS_MODEL.md

Conceptual model for user account, employee profile, linkage, and access.

markdown/03_CONCRETE_DATA_MODEL_AND_BACKEND_SCHEMA.md

Backend-ready table, state, and permission-resolution guidance.

markdown/04_COPILOT_IMPLEMENTATION_PROMPT.md

Direct repo prompt for coding-agent execution.

markdown/05_MASTER_PLAN_IDENTITY_EMPLOYMENT_SEPARATION.md

Master-plan drop-in section for the core architecture phase.

markdown/06_DB_MIGRATION_AND_COMPATIBILITY_ROLLOUT.md

Migration-first rollout rules and compatibility strategy.

markdown/07_FRONTEND_BOOTSTRAP_CONTEXT_SWITCHING_AND_FIELD_OWNERSHIP_UX.md

Frontend context/bootstrap and ownership UX rules.

markdown/08_INVITE_CLAIM_AND_LINK_WORKFLOW.md

Employer-first onboarding and secure claim flow.

markdown/09_RBAC_INTEGRATION_AND_EFFECTIVE_PERMISSION_RESOLVER.md

Employee-linked RBAC resolution and authorization model.

markdown/10_EMPLOYEE_LIFECYCLE_TRANSITIONS_AND_ACCESS_EFFECTS.md

Lifecycle statuses, transitions, and permission consequences.

markdown/11_SELF_SERVICE_VS_EMPLOYER_MANAGED_FIELD_PERMISSIONS.md

Field ownership matrix and self-service boundaries.

markdown/12_MULTI_EMPLOYMENT_MULTI_BUSINESS_AND_TRANSFER_EDGE_CASES.md

Edge-case rules for multiple businesses, scopes, and transfers.

markdown/13_AUDIT_EVENT_LOG_AND_HISTORICAL_TRACEABILITY.md

Audit/event design and historical reconstruction rules.

markdown/14_API_CONTRACT_AND_ENDPOINT_INVENTORY.md

Canonical endpoint inventory and contract guidance.

markdown/15_DATABASE_SCHEMA_DICTIONARY_AND_TABLE_CONTRACTS.md

Table-by-table schema dictionary and enum/index notes.

markdown/16_SERVICE_LAYER_ARCHITECTURE_AND_AUTHORIZATION_HELPER S.md

Service boundaries, actor context, and authorization helpers.

markdown/17_CONCRETE_IMPLEMENTATION_ROADMAP.md

Practical phased rollout order.

markdown/18_CONSOLIDATED_MASTER_SPEC.md

Single-file combined reference.

markdown/19_COPILOT_EXECUTION_SEQUENCE.md

Atomic build slices for Copilot or manual execution.

Architecture Summary

The package standardizes a model where personal platform identity is independent from employer-controlled workforce identity. A User is the login principal. An EmployeeProfile is a business-owned workforce record. A UserEmployeeLink is the bridge that determines whether an authenticated user is linked to that workforce record. EmployeeRoleAssignment attaches scoped RBAC grants to the employment context, not only to the raw user identity.

That design prevents a frequent failure mode in workforce systems: treating the login account and the employer-owned employee file as the same thing. It also makes rehire, transfer, employer-first onboarding, multi-business affiliation, self-service boundaries, and access revocation much easier to reason about.

Core entities

Entity	Owner	Role in system
User	User / platform	Authentication principal, personal account, preferences, security state.
EmployeeProfile	Business	Workforce record, lifecycle status, org placement, operational identity.
UserEmployeeLink	System workflow	Relationship state that controls whether access may flow.
EmployeeRoleAssignment	Business	Business/location role grants attached to employment context.
Effective Access Context	Derived	Resolved permission map used by APIs and frontend bootstrap.

Access activation rule

Workforce access should exist only when four conditions align: active user, active link, access-granting employee state, and active employee-scoped role assignment in the requested tenant scope. That same rule also makes revocation predictable: suspend the link, terminate the employee state, or deactivate the assignment, and the permission disappears on the next authoritative access check.

How to Apply This in the Repo

Best practical sequence:

Phase	Primary deliverable
1	Schema foundations: enums, tables, indexes, constraints.
2	ORM models and relationships.
3	Service scaffolding, actor context, domain errors, audit helper skeleton.
4	Employer-managed employee profile CRUD.
5	Link lifecycle services and endpoints.
6	Employee-linked RBAC assignment services and endpoints.
7	Effective access resolver and permission helpers.
8	`GET /me/access-context` bootstrap API.
9	Lifecycle transition workflows.
10	Self-service and field ownership endpoints.
11	Audit/history query surfaces.
12	Frontend context switching and permission-gated navigation.
13	Legacy compatibility cleanup and hardening.

What not to do

Do not collapse user and employee into one record. Do not grant workforce access directly from ambiguous user-level role shortcuts where employment context is required. Do not let self-service forms mutate employer-owned workforce fields. Do not let lifecycle transitions happen as casual freeform status edits without service-layer rules, audit writes, and access recalculation.

What this package is best for

This package is optimized for planning, architecture alignment, and implementation handoff. It is especially suitable for updating `HN3T\_MASTER\_PLAN.md`, generating Copilot tasks, building migrations and services in reviewable slices, and keeping tenant safety and ownership boundaries intact while the backend and frontend evolve together.